

Audion Office OCR AI

English version

Портативная Windows-платформа для OCR и офисной конверсии. Сканы сохраняются в независимый **DocumentModel**, из которого локально собираются редактируемые и архивные форматы без повторного платного OCR.

Главная идея проекта:

исходники -> OCR Brick -> DocumentModel -> проверка/исправления -> независимые экспортёры

Markdown остаётся удобным контрольным и LLM-форматом, но не является центром OCR-системы.

Независимый DocumentModel

Для каждого документа сохраняется пакет **<имя>.document**: точная копия исходника, изображения всех страниц до и после препроцессинга, хэши, текст и координаты всех OCR-кандидатов, порядок чтения, таблицы и объединения, confidence, сверка Mistral/Tesseract, сырой ответ провайдера и место для ручных исправлений. Полнота означает отсутствие потерь исходных данных, а не обещание 100% точности OCR.

В основном OCR-интерфейсе из пакета независимо и одновременно собираются восемь результатов: **DOCX**, **XLSX**, **Searchable PDF**, **ODT**, **Markdown**, **OCR JSON**, **HTML** и **Проверка**. Рабочий дефолт — только DOCX; форматных пресетов нет. Full archive ZIP остаётся backend-возможностью для совместимости, но не показывается среди основных чипов. Повторная сборка выполняется локально; LibreOffice для неё не требуется. Максимальный размер страницы DOCX — A3.

Для Mistral OCR 4 в GUI доступен независимый второй проход: **Нет**, **Tesseract**, **Yandex** или **Yandex + Tesseract**. Yandex получает тот же очищенный растр независимо от текста Mistral. Для линованных таблиц backend строит физическую сетку по длинным линиям растра и

проверяет число столбцов строкой нумерации `1...N` из координатных OCR-слов. После этого Mistral, Yandex и Tesseract сравниваются уже внутри точных физических ячеек. Объединения восстанавливаются только в шапке; расхождения цифр сохраняются для проверки.

Канонические названия Workbench

Адресные строки называются `Источник` и `Назначение`. Кнопки имеют одинаковые названия во всех NiceGUI-проектах Audion: `Источник`, `Добавить файл...`, `Назначение`, `Сбросить`, `Удалить`, `Список`. `Источник` может быть отдельным файлом или папкой; внешний путь зеркалируется в управляемый `input`, а результаты синхронизируются в выбранное `Назначение`.

CLI/FZF и GUI используют один manifest/service-слой через `system_core/cli_operation.py`: OCR Brick, Workbench resolver и офисные сборщики не имеют отдельных копий backend-логики.

Текущее Состояние

- Английский CLI/FZF-лаунчер остается стабильным backend-интерфейсом: `launcher_project.cmd`.
- Русский CLI/FZF-лаунчер находится рядом: `launcher_project_ru.cmd`.
- NiceGUI-shell доступен как portable-интерфейс оператора: `launcher_gui.cmd`.
- Локальная экстракция находится в `system_core/extract_to_md.py`; новый OCR Brick живёт в `system_core/ocr_brick`.
- Сборщики Markdown уже есть для `DOCX`, Word-based `PDF`, `PPTX` и `XLSX`.
- Отдельный DEV Markdown PDF-движок собирает парные dark/light PDF через Playwright Chromium.
- OCR Brick объединяет препроцессинг сканов, локальный OCR, API OCR, one-page quality test и проверку юридических/финансовых реквизитов.
- Главная продуктовая цель: законченный table-first цикл для PDF/Image-сканов, особенно координатных, числовых и юридических таблиц.

Что Умеет

- `DOCX`, `XLSX`, `PPTX`, `PDF`, `TXT`, `CSV`, `HTML` -> Markdown.
- `PDF`, `JPG`, `PNG`, `TIFF`, `WEBP` -> `DocumentModel` через локальный OCR или API OCR, затем сразу в выбранные форматы.

- Локальный OCR: Tesseract recommended; Surya optional для медленных бесплатных прогонов.
- API OCR: Yandex, Mistral OCR 4, xAI, Gemini и OpenAI через единый OCR Brick contract.
- Gemini поддерживает **Standard**, **Flex** и отдельный **Gemini Batch OCR** для больших несрочных прогонов.
- One-page OCR quality test сравнивает raw/clean и локальные/API-движки перед массовым запуском.
- Проверка реквизитов ищет точные номера контрактов, ИКЗ, даты, суммы и похожие юридические/финансовые строки.
- Извлечение Markdown-таблиц в Excel.
- Выгрузка координатных/табличных проверок в Excel.
- Сборка Markdown в офисный DOCX, плотный LLM-DOCX, PDF через Word, PPTX и XLSX.
- DEV Markdown PDF собирает парные **dark** и **light-sand** PDF из выбранных Markdown-документов через Chromium: умеет искать проектную документацию, пересобирать существующие PDF-пары, обрабатывать только устаревшие PDF и писать результат в подпапку **PDF** рядом с папкой Markdown, прямо рядом с Markdown, в общую **docs/PDF** или зеркалом в текущее Назначение Workbench.
- GUI-поля **Источник** и **Назначение** с рекурсивным зеркалированием:
 - внешний источник -> управляемый **input**
 - **output** -> внешнее назначение
- Машинная отчётность в **report**, отдельно от пользовательского **output**.

Быстрый Старт

Собрать или обновить portable-окружение:

```
builder_main.cmd
```

или напрямую:

```
install\Build_Portable_Env_Build.cmd
```

Build/install-скрипты сохраняют структуру portable GUI template, но добавляют optional component entry points: Playwright Chromium для DEV Markdown PDF, portable Tesseract, Real-ESRGAN ncnn-vulkan и optional Surya acceleration.

Запустить GUI:

```
launcher_gui.cmd
```

Запустить CLI/FZF-лаунчер:

```
launcher_project.cmd
```

Запустить русский CLI/FZF-лаунчер:

```
launcher_project_ru.cmd
```

Первые пункты CLI/FZF соответствуют современным GUI-сценариям: офисный текст без OCR, Tesseract/Surya, Yandex/Mistral/xAI/Gemini/OpenAI, локальный Workbench, OpenAI/Gemini resolver, координаты и проверяемый Mistral/Yandex/Tesseract fusion. Все они идут через `system_core/cli_operation.py` и тот же `office_service.py`.

Типовой сценарий:

1. Положить документы в `input` или выбрать внешний `Источник` в GUI.
2. Для текстовых офисных файлов выбрать `Офисный текст без OCR`.
3. Для сканов сначала запустить `Тест качества OCR` на одной странице.
4. Выбрать `Локальный OCR` или `Платный API OCR`, профиль очистки и модель.
5. Для юридических документов дополнительно запустить `Проверка реквизитов`.
6. В `Готовые файлы` оставить DOCX или одновременно выбрать XLSX, Searchable PDF, ODT и форматы аудита.
7. Проверить готовые DOCX/XLSX в `output` и машинные отчёты в `report`.
8. Использовать `COPY output TO input` и старые Markdown-компиляторы только для архивных compatibility-сценариев.
9. Использовать `DEV Markdown PDF`, когда нужны читаемые dark/light PDF-копии Markdown-документации.

GUI

GUI является оболочкой над существующим ядром, а не отдельной реализацией OCR.

В GUI сейчас есть:

- выбор **Источник** и **Назначение** с picker-кнопками
- рекурсивное зеркалирование входа и выхода
- live terminal log
- root-окна **Офисный текст без OCR**, **Локальный OCR**, **Платный API OCR**, **Тест качества OCR**, **Проверка реквизитов**, **Таблицы**, **Инструменты проекта**
- выпадающие списки моделей и локальных key-файлов для API-провайдеров
- online refresh списка моделей с кэшем для OpenAI, Gemini, Yandex и xAI
- pinned model dropdowns для curated-наборов моделей
- OCR Brick: Tesseract, Surya, Yandex, Mistral OCR 4, xAI, Gemini и OpenAI
- компактный выбор платного OCR-движка в одну строку: один заголовок **Движок**, затем Yandex, xAI, Mistral OCR 4, Gemini и ChatGPT; при очень узком окне строка прокручивается горизонтально, а не распадается на случайные ряды
- общий блок **Готовые файлы** сразу под движком в локальном и платном OCR: затемнённая сетка 4×2 из прямоугольных checkbox-чипов; офисная строка — DOCX/XLSX/Searchable PDF/ODT, dev-аудит — Markdown/OCR JSON/HTML/Проверка; без пресетов, рабочий дефолт — **DOCX**, внутренний DocumentModel сохраняется всегда
- повторные подписи полей **Выходные форматы** и **Очистка скана** скрыты; заголовки секций и нижние поясняющие hint-блоки сохранены
- **Текст + боксы** и **Раскладка** равномерно делят всю строку контракта OCR на две колонки
- подтверждаемая служебная кнопка **Очистить DocumentModel** удаляет ***.document**, ***.document.json** и ***.verification.json**, сохраняя DOCX/XLSX/Markdown/PDF, изображения, исходники и full archive ZIP
- старые Markdown-компиляторы убраны с главного экрана и сохранены в **Инструменты проекта -> Пересборка старого Markdown** только для compatibility с архивными материалами
- Mistral OCR 4 разворачивает табличные attachments в самодостаточный Markdown и безопасно исправляет подтверждённый CP1251/UTF-8 mojibake; для плотных составных листов таблицу всё равно нужно проверять через **Тест качества OCR** или локальный Tesseract-sidecar
- выходы OCR Brick выбираются одновременно checkbox-чипами (**OCR JSON**, **Markdown**, **Проверка**); Mistral + 2-pass Tesseract формирует отдельный verification JSON/Markdown со статусом **pass/review**, согласием чисел и перечнем расхождений
- Gemini **Standard/Flex** и **Gemini Batch OCR** / **Забрать Gemini Batch**

- 2-pass Tesseract как локальная подсказка/sidecar для платных OCR
- окно **Таблицы** без child-навигации: три сразу раскрытых блока для координатных строк, проверки Markdown-таблиц и XLSX-сборки
- DOCX-настройки: книжная/альбомная ориентация, поля в миллиметрах, системный шрифт, размер основного шрифта в pt
- DEV Markdown PDF-настройки: что обработать, где искать, документные пресеты, собственный фильтр имён, список найденных Markdown, режим вывода, а в дополнительных настройках - поля страницы, служебный резерв сверху/снизу и межстрочный интервал
- быстрый доступ к **logs**, **report**, **config**

Команды с параметрами открывают финальный экран: **Назад** слева, запуск справа. Быстрые команды без параметров могут запускаться сразу.

Служебные JSON/XLSX/preview-артефакты пишутся в **report**. Пользовательский **output** должен оставаться местом для результатов, которые можно отдавать дальше.

OCR Brick

Основные OCR-пути:

- локальная экстракция для текстовых офисных форматов
- локальный Tesseract OCR с препроцессингом и координатами
- optional Surya OCR для случаев, когда API-деньги важнее времени
- Yandex Vision OCR с режимами **page**, **table**, **markdown**, **handwritten**
- xAI vision OCR с curated/pinned моделями: **4.20 fast** для буквального OCR и **4.20 quality** reasoning для сложных таблиц и фрагментированных сканов
- обычный non-streaming **chat/completions**, до четырёх попыток с backoff на временных HTTP/сетевых ошибках, строгая проверка UTF-8 и возобновляемый постраничный кэш
- управляемая русская OCR-проверка: по умолчанию выключена; после включения reasoning-модель перепроверяет только подозрительные страницы, чрезмерные правки отклоняются, исходный текст сохраняется в метаданных
- Gemini/OpenAI vision OCR для сравнения качества и сложных страниц
- one-page raw/clean benchmark перед массовым прогоном
- numeric/legal requisites checker для точных строк

Препроцессинг управляется профилем **Очистка скана**: **Авто**, **Без очистки**, **Тяжёлый скан**, **Цифры**, **Ручной**. Подробные ручные параметры спрятаны в Advanced.

Gemini modes documented in `docs/GEMINI_BATCH_FLEX.md`: `Standard` for interactive OCR, `Flex` for cheaper but less predictable synchronous calls, and `Batch` for large non-urgent jobs.

Что ставить для Surya/ускорения

Подробная таблица есть в `install/README_INSTALL.md`.

Короткий выбор:

- RTX 50xx: `[11] REAL-ESRGAN VULKAN`, `[13] SURYA CPU OCR`, `[15] LLAMA.CPP CUDA 13.3`; `[17] SURYA PYTORCH CUDA 50XX` только для тяжёлого benchmark.
- RTX 40xx: `[11]`, `[13]`, `[14] LLAMA.CPP CUDA 12.4`; `[16] SURYA PYTORCH CUDA 40XX` только для benchmark.
- AMD / Intel Arc / Vulkan GPU: `[11]`, `[13]`, `[12] LLAMA.CPP VULKAN`.
- CPU-only: `[13]` только если нужна Surya; чаще разумнее Tesseract или API OCR.

`REAL-ESRGAN VULKAN` улучшает изображение перед OCR, но не распознаёт текст. `SURYA CPU OCR` ставит саму Surya. `LLAMA.CPP` - лёгкий backend ускорения Surya. `SURYA PYTORCH CUDA` - тяжёлый optional benchmark, не базовый путь.

Optional Portable Tesseract

`pytesseract` устанавливает только Python-обвязку. Нативный Tesseract не входит в core bundle. Проект запускается без него; локальный Tesseract OCR и 2-pass проверки будут недоступны, пока Tesseract не установлен.

Если нужен локальный OCR, установите Tesseract как optional portable component. Проект сначала ищет его внутри portable runtime:

```
runtime\tesseract\tesseract.exe
runtime\tesseract\tessdata\eng.traineddata
runtime\tesseract\tessdata\rus.traineddata
runtime\tesseract\tessdata\deu.traineddata
runtime\tesseract\tessdata\osd.traineddata
```

Установите или обновите optional component через `builder_main.cmd` -> `Install portable Tesseract`, или запустите:

```
install\Install-Portable-Tesseract.cmd
```

Свежая core-сборка пересоздаёт `runtime\`; после неё запустите optional component installer ещё раз, если нужен локальный Tesseract OCR.

Установщик владеет только проектной папкой `runtime\tesseract`: при каждом запуске собирает свежий временный payload, скачивает выбранные языковые данные, удаляет предыдущую проектную копию, если она есть, и переносит новую копию на место.

Системный Tesseract не нужен. Если системного source нет, скрипт находит latest stable x64 build UB Mannheim, запускает NSIS installer silent в project staging и затем собирает portable-копию проекта. Если Windows-установка уже есть, например `C:\Program Files\Tesseract-OCR`, путь по умолчанию копирует из неё как из read-only source, чтобы не трогать NSIS maintenance UI. Не указывайте проектную папку руками в UI UB Mannheim installer; целевая папка проекта фиксирована: `runtime\tesseract`.

Если Tesseract лежит в нестандартной локальной папке, задайте `AUDION_TESSERACT_SOURCE_DIR` перед запуском установщика.

Порядок поиска:

1. `runtime\tesseract\tesseract.exe`
2. `AUDION_TESSERACT_EXE`
3. системный `PATH`

Проверка:

```
runtime\tesseract\tesseract.exe --version
runtime\tesseract\tesseract.exe --list-langs
```

Для русско-английских документов используйте язык:

```
rus+eng
```

Если Tesseract не найден, локальный Tesseract OCR и 2-pass режимы должны явно сообщить об ограничении, а не падать молча.

AI Настройки

OCR управляется через:

- `config/llm_settings.yaml`
- `config/ocr_prompt_vision_ocr.md`
- `config/gui_model_pins.json`
- `system_core/ocr_brick/preprocess.profiles.json`

Ключи по умолчанию:

- OpenAI: `config/api_key_openai.txt`
- Gemini: `config/api_key_gemini.txt`
- Yandex: `config/api_key_yandex_studio.txt`
- xAI: `config/api_key_xai.txt`
- Mistral: `config/api_key_mistral.txt`

Дополнительные key-файлы можно хранить в:

- provider-specific extra key files where enabled by the GUI

GUI показывает только путь/статус key-файла, а не значение ключа.

Сборщики Документов

DOCX-сборка поддерживает:

- книжную и альбомную ориентацию
- поля страницы в миллиметрах
- основной шрифт из системного списка
- размер основного шрифта в pt
- обычный офисный DOCX
- плотный LLM-DOCX

PDF имеет два отдельных пути:

- Word-based PDF через `compile_md_to_pdf.py` для офисного результата.
- DEV Markdown PDF через `dev_markdown_pdf_engine.py` для парных dark/light PDF документации через Chromium.

DEV Markdown PDF в GUI работает как управляемый конвейер документации:

- **Что обработать**: Markdown из **input/output**, файл или папку из поля **Источник**, документацию по указанному пути, существующие PDF-пары или только устаревшие PDF.
- **Где искать**: относительный путь от корня проекта или абсолютный путь; пустое поле означает корень текущего проекта.
- **Документные пресеты**: **README**, **USER**, **GUIDE**, **GUI**, **GUARD**, **ARCHITECTURE**, **CHANGELOG**, **INSTALL**, **PORTABLE**, **USAGE**, **CONFIG**, **TROUBLE**, **SECURITY** и дополнительные отключённые пресеты вроде **FAQ**, **API**, **ROADMAP**, **MIGRATION**, **RELEASE**, **LICENSE**, **AGENTS**.
- **Свой фильтр имён**: фрагменты имени или пути через запятую, например **API**, **ROADMAP**, **INSTALL**.
- **Найденные Markdown**: кнопки **Сканировать**, **Выбрать все**, **Только без PDF**, **Только устаревшие**, **Только CHANGELOG**, **Снять все**.
- **Режим вывода**: **docs/PDF** по умолчанию; также доступны **PDF рядом с MD**, прямо рядом с Markdown или зеркало в текущее Назначение Workbench.
- После сборки в терминале выводится краткий отчёт: **MD**, **PDF**, **Pages**, **Errors**, **Stale after export**.

XLSX можно собрать прямо из сохранённого DocumentModel без повторного OCR. Для линованных сканов backend консервативно восстанавливает физическую сетку по линиям раstra, сверяет число столбцов с координатной строкой **1...N**, назначает текст провайдеров подтверждённым ячейкам и оставляет неоднозначные либо числовые расхождения в проверке. Root-окно **Таблицы** дополнительно содержит три раскрытых служебных блока: координатные строки, проверку Markdown-таблиц и legacy XLSX-сборку.

Структура Проекта

```
Audion Office OCR AI/
├─ config/                # API keys, OCR prompts, LLM/OCR settings, GUI settings
├─ data/                  # model-list cache and other local cache files
├─ input/                 # staging-зона исходников
├─ output/                # пользовательские результаты
├─ report/                # машинные отчёты и служебные артефакты
├─ logs/                  # журналы выполнения
├─ install/               # сборка, установка, release scripts
├─ system_core/           # Python-ядро OCR, OCR Brick, GUI services
├─ runtime/               # portable Python и optional component runtime, создается
                           локально
├─ wheelhouse/            # offline wheels, создается локально
├─ release/               # release archives
├─ launcher_gui.cmd       # GUI launcher
├─ launcher_project.cmd   # EN CLI/FZF launcher
├─ launcher_project_ru.cmd # RU CLI/FZF launcher
├─ launcher_tools.cmd     # tools, build, diagnostics
└─ builder_main.cmd       # portable environment builder
```

Безопасность

Локальная экстракция, сборка и локальный OCR работают с файлами на машине. Локальный Tesseract использует optional portable Tesseract из `runtime\tesseract`, если он установлен. DEV Markdown PDF рендерится локально через portable Chromium. API OCR отправляет изображения, отрендеренные страницы и, если включён 2-pass, локальные OCR-подсказки выбранному провайдеру. Gemini Batch сначала пишет локальный JSONL/manifest; отправка в Google происходит только при включённом переключателе отправки.

Используйте API OCR только там, где политика разрешает передачу изображений документов выбранному провайдеру.

Не коммитьте:

- API keys
- `runtime/`
- `wheelhouse/`
- `output/`

- [report/](#)
- [logs/](#)
- [release/](#)
- [workspace/](#)
- [data/](#)

Roadmap

- Глубже сегментировать таблицы и table-crop зоны в OCR Brick.
- Развить quality/requisites отчёты в guided review/edit workflow.
- Добавить более сильную проверку координатных таблиц перед XLSX.
- Сохранить CLI/FZF как стабильный backend.
- Держать EN/RU-лаунчеры в паритете.
- Держать GUI как тонкую portable-оболочку над проверяемыми командами.

Лицензии

Сведения о сторонних лицензиях см. в каталоге [licenses/](#) внутри релизного пакета.